Integración MagicBridge con LabVIEW

📋 Resumen Ejecutivo

Este documento describe cómo integrar el MagicBridge ELMOS Programmer con LabVIEW en Windows para automatización de producción.

Arquitectura de Integración

```
┌─────────────────────────────────────────────────────────────┐
│                        LabVIEW                              │
│                    (Interfaz Principal)                     │
└─────────────────────────┬───────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│              System Exec / Python Node                      │
│           (Llamadas a scripts Python)                       │
└─────────────────────────┬───────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│           Scripts Python MagicBridge                        │
│    (1_cargar_datos_v3.py / 2_ejecutar_comando_v3.py)        │
└─────────────────────────┬───────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│                  Puerto Serial USB                          │
│                (COM3, COM4, etc.)                           │
└─────────────────────────┬───────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│                  MagicBridge Hardware                       │
│              (ELMOS Programmer)                             │
└─────────────────────────────────────────────────────────────┘
```

🎯 Opciones de Integración

✅ Opción 1: System Exec (RECOMENDADA)

Descripción:
Ejecutar los scripts Python desde LabVIEW usando el VI "System Exec.vi"

Ventajas:

- ✅ Simple de implementar
- ✅ No requiere modificaciones en scripts Python
- ✅ Fácil de depurar
- ✅ Compatible con todas las versiones de LabVIEW

Desventajas:

- ⚠️ Requiere parsear salida de texto
- ⚠️ No es la opción más rápida

Ideal para:

- Automatización de producción
- Estaciones de test
- Proyectos que ya usan System Exec

✅ Opción 2: Python Node (LabVIEW 2018+)

Descripción:
Usar el Python Node integrado en LabVIEW 2018 y superiores

Ventajas:

- ✅ Integración nativa
- ✅ Paso directo de datos
- ✅ Mejor rendimiento
- ✅ Manejo de errores más robusto

Desventajas:

- ⚠️ Solo LabVIEW 2018+
- ⚠️ Requiere instalar Python Integration Toolkit
- ⚠️ Requiere adaptar scripts ligeramente

Ideal para:

- Instalaciones modernas de LabVIEW
- Aplicaciones complejas
- Máxima integración

✅ Opción 3: DLL Wrapper (Avanzada)

Descripción:
Crear DLL de Windows que envuelva los scripts Python

Ventajas:

- ✅ Máxima velocidad
- ✅ Oculta completamente la implementación Python
- ✅ API nativa para LabVIEW

Desventajas:

- ⚠️ Más complejo de implementar
- ⚠️ Requiere recompilar para cambios
- ⚠️ Mantenimiento más complejo

Ideal para:

- Productos comerciales
- Máxima velocidad requerida
- Ocultar implementación completamente

🚀 Implementación Opción 1: System Exec (Paso a Paso)

Requisitos Previos

1. **LabVIEW instalado** (cualquier versión)
2. **Python 3.6+ instalado en Windows**
   - Descargar de: https://www.python.org/downloads/
   - ⚠️ IMPORTANTE: Marcar "Add Python to PATH" durante instalación
3. **pyserial instalado**

   ```
   pip install pyserial
   ```

4. **Scripts MagicBridge** en una carpeta conocida

Paso 1: Configuración Inicial

1. Copiar la carpeta `MagicBridge_Client_Package` a:

   ```
   C:\MagicBridge\
   ```

2. Verificar que Python funciona:

```
python --version
python C:\MagicBridge\scripts\1_cargar_datos_v3.py --help
```

Paso 2: Crear VI de Carga de Datos

Archivo: `MagicBridge_Load.vi`

Controles (Inputs):

- `HMF File Path` (String) - Ruta al archivo HMF
- `COM Port` (String) - Puerto COM (ej: "COM3")
- `PCB Number` (Numeric) - Número de PCB (3, 5, 6)
- `Variant` (String) - Variante ("D" o "E")

Indicadores (Outputs):

- `Success` (Boolean) - True si exitoso
- `Output` (String) - Salida del comando
- `Error Code` (Numeric) - Código de error (0 = éxito)
- `JSON Result` (String) - Resultado en formato JSON

Diagrama de bloques:

```
[HMF File Path] ────┐
[COM Port] ─────────┤
[PCB Number] ───────┤──> [Build Command] ──> [System Exec.vi] ──> [Parse Output] ──> [Success]
[Variant] ──────────┘                                                                  │
                                                                                       ├──> [Output]
                                                                                       ├──> [Error
Code]
                                                                                       └──> [JSON
Result]
```

Command String Builder:

```
python C:\MagicBridge\scripts\1_cargar_datos_v3.py [HMF File Path] [COM Port] [PCB Number]
[Variant] --json C:\Temp\result.json --quiet
```

Ejemplo de comando generado:

```
python C:\MagicBridge\scripts\1_cargar_datos_v3.py C:\Data\archivo.hmf COM3 5 D --json
C:\Temp\result.json --quiet
```

Paso 3: Crear VI de Ejecución de Comandos

Archivo: `MagicBridge_Command.vi`

Controles (Inputs):

- `COM Port` (String) - Puerto COM
- `Command` (String) - Comando (ej: "W5d", "R3e", "P6d")

Indicadores (Outputs):

- `Success` (Boolean)
- `Output` (String)
- `Error Code` (Numeric)
- `JSON Result` (String)

Command String Builder:

```
python C:\MagicBridge\scripts\2_ejecutar_comando_v3.py [COM Port] [Command] --json
C:\Temp\cmd_result.json --quiet
```

Paso 4: Parsear Resultados JSON

LabVIEW puede parsear JSON de varias formas:

Opción A: Leer archivo JSON y parsear manualmente

```
[Read File] → [String to JSON] → [Parse Keys]
```

Opción B: Usar JKI JSON Toolkit (Recomendado)

- Instalar desde VI Package Manager
- Proporciona VIs para parsear JSON fácilmente

Opción C: Parsear como texto

- Buscar "status": "SUCCESS" en el output
- Extraer "exit_code": 0

Paso 5: VI Principal de Integración

Archivo: `MagicBridge_Main.vi`

Flujo:

```
[START]
   |
   ▼
```

```
[Detectar COM Port] ──────┐
   │                      │
   ▼                      │
[Cargar Datos HMF] ────────┤
   │                      │ Error? → [Show Error]
   ▼                      │
[Loop: Programar Chips]    │
   │                      │
   ├─> [Ejecutar W5d] ─────┤
   │                      │
   ├─> [Verificar] ────────┤
   │                      │
   └─> [Siguiente Chip]    │
                          │
[Resumen y Estadísticas] <─┘
```

📝 Código de Ejemplo LabVIEW (Pseudocódigo)

Ejemplo 1: Cargar Datos

```
// Construir comando
comando = "python C:\MagicBridge\scripts\1_cargar_datos_v3.py " +
          filepath + " " +
          comport + " " +
          String(pcb) + " " +
          variant +
          " --json C:\Temp\result.json --quiet"

// Ejecutar
[System Exec.vi]
  - command line: comando
  - wait until completion: TRUE
  - standard output: output_str
  - return code: exit_code

// Verificar resultado
IF exit_code == 0 THEN
    // Leer JSON
    json_str = Read File("C:\Temp\result.json")

    // Parsear (ejemplo simple)
    IF Contains(json_str, "\"status\": \"SUCCESS\"") THEN
        resultado = TRUE
    ELSE
        resultado = FALSE
    END IF
ELSE
    resultado = FALSE
```

```
END IF
```

Ejemplo 2: Programar Chip

```
// Construir comando
comando = "python C:\MagicBridge\scripts\2_ejecutar_comando_v3.py " +
          comport + " " +
          command +
          " --json C:\Temp\cmd.json --quiet"

// Ejecutar
[System Exec.vi]
  - command line: comando
  - return code: exit_code

// Verificar
IF exit_code == 0 THEN
    success = TRUE
ELSE
    success = FALSE
END IF
```

Ejemplo 3: Loop de Producción

```
// Cargar datos UNA VEZ
CargarDatos(hmf_file, "COM3", 5, "D")

// Loop de producción
WHILE operador_continua DO
    // Esperar chip nuevo
    MostrarMensaje("Coloque chip y presione continuar")

    // Programar
    resultado = EjecutarComando("COM3", "W5d")

    IF resultado == TRUE THEN
        contador_ok = contador_ok + 1
        EncenderLED_Verde()
    ELSE
        contador_error = contador_error + 1
        EncenderLED_Rojo()
    END IF

    // Actualizar pantalla
    MostrarEstadisticas(contador_ok, contador_error)
END WHILE
```

🔧 Configuración de LabVIEW

Configurar System Exec.vi

1. **Timeout:** Aumentar a 300000 ms (5 minutos)
    - La carga de datos puede tardar 10-20 segundos
    - La programación tarda 5-10 segundos
2. **Working Directory:** Establecer a `C:\MagicBridge\scripts\`
    - Ayuda si hay rutas relativas
3. **Wait Until Completion:** TRUE
    - Asegura que LabVIEW espere a que termine
4. **Standard Output:** Conectar siempre
    - Útil para debugging

Manejo de Errores

```
[System Exec.vi] → [Return Code]
                            │
                            ├─ 0: Éxito
                            ├─ 1: Error argumentos
                            ├─ 2: Error archivo HMF
                            ├─ 3: Error conexión
                            └─ 4: Error operación

[Case Structure basado en Return Code]
  Case 0: Continue
  Case 1: "Verificar argumentos del comando"
  Case 2: "Verificar archivo HMF existe"
  Case 3: "Verificar MagicBridge conectado al puerto COM"
  Case 4: "Error en programación - Revisar chip"
```

📊 Interfaz de Usuario Recomendada

Panel Frontal

```
┌──────────────────────────────────────────────────────────────┐
│  MagicBridge ELMOS Programmer                      [X]       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Configuración:                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Archivo HMF: [C:\Data\archivo.hmf    ] [Browse...]    │    │
│  │ Puerto COM:  [COM3 ▼]                                 │    │
│  │ PCB:         [5 ▼]      Variante: [D ▼]               │    │
```

```
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  Control:                                                  │
│  ┌──────────────────────────────────────────────────────┐  │
│  │ [Cargar Datos]  [Programar Chip]  [Solo Leer]         │  │
│  │                                                       │  │
│  │ Estado: ●●● LISTO                                     │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  Estadísticas:                                             │
│  ┌──────────────────────────────────────────────────────┐  │
│  │ Chips OK:    [  15  ]    Chips Error: [  2  ]         │  │
│  │ Tasa éxito:  [ 88%  ]    Tiempo prom: [ 8.5s ]        │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  Log:                                                      │
│  ┌──────────────────────────────────────────────────────┐  │
│  │ [15:30:22] Datos cargados exitosamente                │  │
│  │ [15:30:25] Chip #15 programado OK                     │  │
│  │ [15:30:28] Chip #16 ERROR - Reintentar                │  │
│  │ [15:30:35] Chip #16 programado OK (reintento)         │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  [Iniciar Modo Automático]  [Detener]  [Salir]             │
└────────────────────────────────────────────────────────────┘
```

🔌 Detectar Puerto COM Automáticamente

Script PowerShell Helper

Crear `detect_magicbridge.ps1`:

```
# Buscar MagicBridge en puertos COM
Get-PnpDevice | Where-Object {$_.FriendlyName -like "*ELMOS*"} |
    Select-Object -ExpandProperty FriendlyName
```

Desde LabVIEW

```
[System Exec.vi]
  Command: powershell -File C:\MagicBridge\detect_magicbridge.ps1
  Output: "LiCore ELMOS Programmer (COM3)"

[String Subset] → Extraer "COM3"
```

Script Python Helper

Crear `detect_port.py`:

```
import serial.tools.list_ports

for port in serial.tools.list_ports.comports():
    if "1d50:6015" in port.hwid.lower() or "elmos" in port.description.lower():
        print(port.device)
        break
```

Desde LabVIEW:

```
[System Exec.vi]
  Command: python C:\MagicBridge\detect_port.py
  Output: "COM3"
```

📁 Estructura de Archivos Recomendada

```
C:\MagicBridge\
├── scripts\
│   ├── 1_cargar_datos_v3.py
│   ├── 2_ejecutar_comando_v3.py
│   └── hmf_loader.py
│
├── data\
│   └── archivo.hmf              (archivos HMF del cliente)
│
├── temp\
│   ├── result.json              (resultados temporales)
│   └── cmd.json
│
├── logs\
│   └── production_YYYYMMDD.log  (logs de producción)
│
└── helpers\
    ├── detect_port.py
    └── detect_magicbridge.ps1
```

⚡ Optimización de Rendimiento

Técnica 1: Reutilizar Datos Cargados

```
// Cargar datos una sola vez al inicio
AL INICIO DE TURNO:
    CargarDatos(hmf_file, "COM3", 5, "D")

// Durante producción (sin recargar)
LOOP:
    EjecutarComando("COM3", "W5d")  // Usa datos ya en RAM
```

Ahorro: ~15 segundos por chip (después del primero)

Técnica 2: Modo Batch JSON

```
// Generar JSONs sin esperar output en consola
comando = "python script.py ... --json result.json --quiet"

// LabVIEW solo lee el JSON al final
json = LeerArchivo("result.json")
```

Ahorro: ~2-3 segundos por operación

Técnica 3: Paralelización (Si múltiples MagicBridge)

```
PARALLEL:
    Thread 1: MagicBridge_1 → COM3 → Chip A
    Thread 2: MagicBridge_2 → COM4 → Chip B
```

Ahorro: 50% del tiempo total (2 chips simultáneos)

🐛 Debugging y Troubleshooting

Problema: "Python no encontrado"

Solución en LabVIEW:

```
// Usar ruta completa de Python
comando = "C:\Users\[User]\AppData\Local\Programs\Python\Python39\python.exe " +
          "C:\MagicBridge\scripts\1_cargar_datos_v3.py ..."
```

Problema: "Timeout en System Exec"

Solución:

1. Aumentar timeout a 300000 ms
2. Verificar que script no esté esperando input
3. Usar `--quiet` para evitar prompts

Problema: "No se puede parsear JSON"

Solución:

```
// Verificar que archivo existe
File Exists? → IF FALSE: Error

// Leer contenido completo
json_string = Read File("result.json")

// Verificar que no está vacío
IF Length(json_string) == 0 THEN
    Error: "Archivo JSON vacío"
END IF
```

Problema: "Puerto COM no disponible"

Solución:

```
// Detectar puerto antes de cada operación
puerto = DetectarPuerto()

IF puerto == "" THEN
    Error: "MagicBridge no conectado"
    Mostrar: "Conecte MagicBridge y presione Retry"
END IF
```

📈 Monitoreo y Logging

Log de Producción en LabVIEW

```
// Formato de log
timestamp + "," + chip_id + "," + resultado + "," + tiempo_ms

// Ejemplo
"2026-03-23 15:30:22,CHIP-001,OK,8500"
"2026-03-23 15:30:35,CHIP-002,ERROR,12000"
"2026-03-23 15:30:50,CHIP-002,OK,9000"
```

```
// Escribir a archivo
Append to File("C:\MagicBridge\logs\production.csv")
```

Dashboard de Estadísticas

```
// Calcular en tiempo real
total_chips = contador_ok + contador_error
tasa_exito = (contador_ok / total_chips) * 100
tiempo_promedio = suma_tiempos / contador_ok

// Mostrar en gráficos
Graph: Chips por hora
Graph: Tasa de éxito vs tiempo
Graph: Tiempo promedio de programación
```

🔒 Consideraciones de Seguridad

Validación de Entradas

```
// Validar puerto COM
IF NOT Match(puerto, "COM[0-9]+") THEN
    Error: "Puerto COM inválido"
END IF

// Validar archivo HMF existe
IF NOT File Exists(hmf_file) THEN
    Error: "Archivo HMF no encontrado"
END IF

// Validar PCB
IF NOT (pcb IN [3, 5, 6]) THEN
    Error: "PCB debe ser 3, 5 o 6"
END IF
```

Manejo de Errores Críticos

```
TRY:
    resultado = EjecutarComando()
CATCH Error:
    Log Error
    Notificar Operador
    IF error_count > 3 THEN
        Detener Producción
        Notificar Supervisor
    END IF
END TRY
```

📚 Recursos Adicionales

Documentos de Referencia

- System Exec.vi Documentation
- JSON Parsing in LabVIEW
- Error Codes Reference

Archivos de Ejemplo

- `MagicBridge_Load.vi` - VI de carga de datos
- `MagicBridge_Command.vi` - VI de comandos
- `MagicBridge_Main.vi` - Aplicación completa
- `Example_Production_Loop.vi` - Loop de producción

Scripts Helper

- `detect_port.py` - Detectar puerto COM automáticamente
- `verify_connection.py` - Verificar conexión MagicBridge
- `test_communication.py` - Test de comunicación

🚀 Checklist de Implementación

- [ ] Python 3.6+ instalado en Windows
- [ ] pyserial instalado
- [ ] Scripts MagicBridge en C:\MagicBridge\
- [ ] VI de carga creado y probado
- [ ] VI de comandos creado y probado

- [ ] Detección de puerto COM implementada
- [ ] Manejo de errores implementado
- [ ] Logging de producción implementado
- [ ] Interfaz de usuario completada
- [ ] Testing con chips reales exitoso
- [ ] Documentación de operador creada

MagicBridge ELMOS Programmer
Guía de Integración con LabVIEW v1.0
© 2026 LiCore